

PROJECT REPORT ON STUDENT MANAGEMENT SYSTEM

A Simple File-Based Python Project for Managing Student Data

Project Title	Student Management System (Python)
Language Used	Python 3 (Basic File Handling)
Storage Type	Text File (`records.txt`)
Submitted By	[Student Name / Roll Number]
Course / Dept	Computer Science / IT

1. Introduction

In this project, I made a simple Student Management System using Python language. The main reason for making this project is to store student details easily without using big database software like MySQL or Oracle. Everything is stored inside a normal text file called `records.txt`.

Normally in colleges or hostels, keeping record of students manually on paper is very confusing and hard. Pages get lost or destroyed. So I made this program where anyone can add student details, search them using their registration number, update their info if changes happen, and delete them if they leave.

2. Purpose of the Project

The main purpose of making this project is listed below:

- To learn how Python handles simple files using `open()`, `readlines()`, and `write()`.
- To break code into small parts using multiple files like `display.py`, `records.py`, etc., so code looks clean.
- To make a terminal-based system which works fast on any normal computer.
- To save data permanently so when program closes, data is not deleted.

Note: I did not use any advanced libraries or external frameworks. Plain simple Python logic with basic loops and functions is used.

3. Files Used in the Project

I divided the project into 7 files so that it doesn't get messed up in one big file. Below is the explanation of all files:

File Name	Description / What it does
main.py	This is the main file. It runs the main loop and imports other modules.
menu.py	It prints the options menu (1 to 6) and takes user input choice.
records.py	Contains functions to add, search, update and delete student details.
file_handler.py	Reads data from <code>records.txt</code> and writes data back when saving.
display.py	Contains code to show output in nice table formatting on screen.
config.py	Stores the filename variable <code>FILENAME = "records.txt"</code> .
records.txt	Text file where all student data is saved line by line.

4. Structure of Data

In the file `records.txt`, I saved student data using vertical bar `|` as a separator. Each row represents one student record:

```
Reg_No | Name | Hostel_Block | State | Email | Contact 101 | suryansh | 612 | up | sury  
ansh@mail.com | 8004108085
```

When program starts, `file_handler.py` reads this text file, splits each line using `.split(" | ")` and converts it into a Python Dictionary so that searching becomes very fast using registration numbers.

5. Features implemented

- **Add Student:** Takes details like reg number, name, hostel, state, email, contact and saves it.
- **Search Student:** Asks for registration number and prints that student's details.
- **Update Student:** Asks for reg number and lets you rewrite the details.
- **Delete Student:** Removes student entry from dictionary and updates file.
- **Show All:** Displays list of all students present in the file.

6. Code Flow & Working Explanation

Here is how the project works step by step when you run `main.py`:

1. Program starts in `main.py`.
2. It immediately calls `file_handler.load_records()` which reads `records.txt` and creates a dictionary in memory.
3. Then a `while True` loop starts and shows menu options via `menu.display_menu()`.
4. User enters choice 1, 2, 3, 4, 5, or 6.
5. According to choice, functions in `records.py` or `display.py` get executed.
6. Every time a record is added, updated or deleted, `file_handler.save_records()` rewrites the `records.txt` file automatically.

Sample Menu Screen Output:

```
===== STUDENT MANAGEMENT MENU =====  
1. Add Student Record 2. Search Student Record 3. Update Student Record 4. Delete Student Record 5. Show All Student Records 6. Exit ===== Enter your choice (1-6):
```

7. Difficulties Faced while Coding

While making this small project, I faced a few problems:

- **Formatting Issue:** Aligning output lines properly on screen was hard. I used formatting like `f"{reg_no:<12}"` in `display.py` to fix alignment.
- **File Reading Errors:** When file was empty or missing, program was crashing. So I added a simple `try ... except` inside `load_records()` so program doesn't crash.
- **String Split Problem:** Forgetting space near `|` caused issues during search. So I made sure to split strictly by `" | "`.

8. How to Run the Project

To run this system, you need Python installed on your computer. Follow these basic steps:

1. Put all files (`main.py`, `records.py`, `file_handler.py`, `display.py`, `menu.py`, `config.py`, and `records.txt`) in one single folder.
2. Open command prompt or terminal inside that folder.
3. Type the following command:

```
python main.py
```

9. Future Improvements

Right now the project is very basic and simple, but in future I can make it better by adding:

- Adding password login system for teachers or admin.
- Using real database like SQLite instead of text file.
- Adding input validation so user cannot enter bad phone numbers or wrong registration formats.
- Creating a Graphical User Interface (GUI) using Tkinter library instead of black terminal screen.

10. Conclusion

This Student Management System project was a good learning experience for me. It helped me understand basic concepts of Python programming like functions, modules, dictionaries, loops, and file handling. It fulfills all fundamental requirements needed to store, view, and modify student information effectively.

Student Signature

Teacher / Guide Signature